Certified Compilation of Graph Coloring to Neutral-Atom Hardware

An Exact Gadgetized Reduction with a Heuristic Layout Optimizer

Research Draft

June 9, 2026

Abstract

Neutral-atom quantum processors solve a single native problem: maximum-weight independent set (MWIS) on a unit-disk graph (UDG), realized physically by the Rydberg blockade. They do *not* color graphs. We present a compilation pipeline that takes an arbitrary finite graph coloring instance and produces a hardware-executable UDG–MWIS instance whose ground state decodes back to a proper coloring of the original graph. The pipeline has two layers with a clean separation of concerns. The *primary path* is an exact, polynomial-time two-stage reduction: (i) a textbook reduction of k-colorability to maximum independent set on an auxiliary graph G' of size nk, followed by (ii) a gadgetized embedding of G' into a weighted UDG using copy and crossing gadgets in the style of Nguyen et al. Correctness here is a once-proved meta-theorem, not a per-instance gamble: there is no chromatic-number inflation and the decoder is an exact inverse. The genuine engineering uncertainty therefore moves entirely out of correctness and into *geometry*: atom count, array area, crossing number, and analog-solve quality. The *secondary path* is a suite of seven polynomial-time geometric heuristics. We prove a hard impossibility result that disqualifies these heuristics as a stand-alone coloring mechanism for general graphs, and we therefore repurpose them as the *layout optimizer* that places and compacts the gadgetized instance under hardware constraints. We give the reductions with proofs, size and feasibility bounds, worked examples, and a complexity analysis of the full pipeline. Beyond the theory we report a working implementation: an axiom-free *Coq/Rocq* mechanization of the certified-reduction calculus (every head theorem closed under the global context, including Stage-1 exactness, the gadget offset invariant, end-to-end pipeline soundness, and a stage-ordering corollary); a prototype typed front end in Haskell (parser, type-and-effect checker, elaborator, device emitter) whose two test suites pass 12,230 checks; and an empirical validation in which a faithful Rydberg adiabatic simulator solves the compiled instances and decodes correct colorings. We validate against a brute-force oracle on a suite of named graphs (18/18 instances matching exactly), break the classical 2^{nk} simulation wall with a certified separator decomposition that keeps every device solve to ≤ 13 atoms while the whole-instance nk reaches 192, and run the canonical **DIMACS Mycielski** benchmark, on which the pipeline certifies the chromatic number exactly in both directions (finding a χ-coloring and *proving* no $(\chi-1)$-coloring exists). Tying these together, we present a *coloring-specification language* and a *certified-reduction type system* whose types are problem reductions rather than data shapes: each compiler stage is typed as an answer-preserving morphism carrying an executable decoder, a single composition rule yields end-to-end soundness, the legal stage order falls out as a typing corollary, the supergraph path is gated by the impossibility bound as a side-condition, and a feasibility refinement and split/stitch decomposition keep device size bounded. The core of this calculus is exactly what the Coq development mechanizes and the Haskell checker implements; growing it into a full surface language with complete elaboration metatheory is the remaining work.

Keywords. graph coloring, unit-disk graphs, maximum independent set, Rydberg blockade, neutral-atom computing, gadget reductions, certified compilation, QuEra, Bloqade.

Contents

1 Introduction

Graph coloring is a canonical NP-complete problem [28, 21] with broad practical reach, from register allocation and scheduling to frequency assignment. A recent and physically distinctive way to attack such combinatorial problems is the neutral-atom quantum platform, in which individually trapped atoms are excited to Rydberg states and interact through the *Rydberg blockade* [42, 25, 43]. The decisive feature of this platform is geometric: two atoms within a blockade radius r cannot both be excited, so the set of simultaneously excited atoms is precisely an *independent set* of the unit-disk graph (UDG) induced by atom positions. The device's analog ground state therefore encodes a *maximum-weight independent set* (MWIS) of that UDG [38, 18].

This single fact frames the entire compilation problem and is the pivot of this paper:

> *The hardware solves MWIS on a unit-disk graph. It does not color. Any pipeline that compiles coloring to this hardware must* reduce *coloring to UDG–MWIS, not merely* embed *the input graph geometrically.*

1.1 Two tempting designs, and why one is correct

There are two natural strategies for getting an arbitrary coloring instance onto this hardware.

Design A — geometric supergraph embedding. Place the original vertices in $\mathbb{R}^2$ and choose a radius r so that the induced UDG H is a *supergraph* of the input G, i.e. $E(G) \subseteq E(H)$. Then any proper coloring of H restricts to a proper coloring of G. This is attractive because the supergraph relation is trivially achievable and gives a one-line decoder. Section 5 develops the seven heuristics that make H as good as possible. However, Design A has two structural defects. First, we prove (Theorem 5.4) that a same-vertex UDG supergraph *cannot* bound chromatic-number inflation in general: the star $K_{1,n}$ has $\chi = 2$ but forces $\chi(H) \geq \lceil n/5 \rceil \to \infty$ in any UDG supergraph. Second, and independently, even a perfect H is not executable, because the device cannot color H; coloring a UDG on an MWIS machine requires iterated independent-set extraction, which is itself heuristic. Design A is thus heuristic twice and unbounded once.

Design B — exact gadgetized reduction. Reduce k-colorability of G to a *single* MWIS instance, then embed that instance into a weighted UDG the hardware can run. This is the standard reduction route, and it composes two exact, polynomial-time steps (Section 4): the classical colorability→independent-set reduction, and the Rydberg gadgetization of an arbitrary graph into a UDG via copy/crossing gadgets [37]. Here correctness is a once-proved meta-theorem with no inflation and an exact inverse decoder. The remaining uncertainty is not about *whether* the answer is correct but about *whether it fits the device and whether the analog dynamics finds the ground state* — both geometric/physical questions.

We adopt Design B as the primary path and retain Design A's heuristics in a strictly subordinate, and now sound, role: the *layout optimizer* that realizes the gadgetized instance under hardware spacing while minimizing atom count, area, and crossings. Because the gadget structure is fixed and exact, the heuristics can no longer affect correctness — only feasibility and solve quality. This relocation is the central design decision of the paper.

1.2 Contributions

(1) A clean statement of the compilation problem that takes the hardware's native primitive (UDG–MWIS) as the target, and a two-layer pipeline that separates exact correctness from heuristic geometry (Sections 3–4).

(2) The exact primary reduction: colorability→MIS with a self-contained correctness proof and size nk (Theorem 4.2), composed with Rydberg UDG gadgetization (Theorem 4.4) to give an end-to-end exactness and decoding meta-theorem (Theorem 4.5), together with $O((nk)^2)$ atom-count and feasibility bounds (Section 4.4).

(3) An impossibility result (Theorem 5.4) that rigorously explains why the geometric-supergraph approach cannot be the correctness mechanism, justifying its demotion.

(4) Seven polynomial-time geometric heuristics, corrected and re-targeted as a hardware-aware *layout optimizer* for the gadget instance, with termination guarantees and a corrected catalogue of provably exact graph families (Section 5).

(5) An integrated pipeline with worked examples and a full complexity accounting (Sections 6–7).

(6) A coloring-specification *language* and a *certified-reduction type system* (Section 8): problem-type sorts indexed by the invariants the reductions preserve, a single decoder-carrying reduction judgment, per-stage typing rules whose composition is the soundness theorem (Theorem 8.3), a corollary that the legal stage order is forced by typing (Corollary 8.4), and refinements for certificate grade, the impossibility-gated supergraph safety effect, and bounded-width decomposition.

(7) A working implementation and a quantitative evaluation (Section 9): (a) an *axiom-free* Coq/Rocq mechanization of the certified-reduction calculus, so Stage-1 exactness, the gadget offset invariant, end-to-end pipeline soundness, and the legal stage ordering are machine-checked theorems; (b) a prototype typed front end in Haskell (parser, type-and-effect checker, elaborator, device emitter) passing 12,230 test checks; and (c) an empirical validation against a brute-force oracle and a faithful Rydberg adiabatic simulator, including a certified separator decomposition that defeats the 2^{nk} classical- simulation wall and a run of the canonical DIMACS Mycielski benchmark on which the chromatic number is certified exactly.

2 Related Work

Graph coloring and its complexity. Deciding k-colorability for $k \geq 3$ is NP-complete [28, 21], and the chromatic number is hard even to approximate. The chromatic number has been studied through the chromatic polynomial since Tutte [45], and deep lower bounds come from topological methods such as Lovász's resolution of the Kneser conjecture [31]; our benchmark family, the Mycielskians (Section 9.5), is the standard source of triangle-free graphs with arbitrarily large χ. Practical colorings are produced by greedy orderings such as largest-first [46] and by the saturation-degree heuristic DSATUR [8]; the clique number gives the basic lower bound $\omega(G) \leq \chi(G)$ [47], and computing it is itself the (NP-hard) maximum-clique problem [6]. We use DSATUR only as an auxiliary *witness* generator in the secondary path, never on the exact path.

Unit-disk graphs. UDGs are the intersection graphs of unit disks in the plane [12]. Recognition — deciding whether a given abstract graph is a UDG — is NP-hard [9, 27], which is precisely why we never attempt exact realization of the input graph. The geometric constraint interacts sharply with coloring: even unit-*distance* graphs in the plane can already force $\chi = 4$ [13], underscoring that distance-based adjacency carries genuine chromatic content. UDGs enjoy strong geometric locality: every clique lies inside a disk of radius r, and the packing of points in such a disk is bounded [26], a fact we use both for the impossibility result and for clique control.

Rydberg MWIS and gadgetized reductions. The identification of the Rydberg blockade ground state with MWIS on a UDG is due to Pichler et al. [38] and was demonstrated at scale on hundreds of atoms by Ebadi et al. [18]. Because native UDGs are geometrically restricted, Nguyen et al. [37] introduced copy and crossing gadgets that encode MWIS on an *arbitrary* graph as MWIS on a weighted UDG laid out on a grid, with a known value offset and a structural decoder. Our primary path is exactly the composition of the classical colorability→MIS reduction with this gadgetization; the hardware target is QuEra's neutral-atom machine [48], programmed through the Bloqade simulation stack. The programmable-Rydberg platform has matured rapidly—from quantum phases in optical lattices [17] to multi-qubit gate-model processors [24]—and competing quantum hardware such as photonic systems [34] targets the same optimization problems by different physical means; our compilation strategy is, however, specific to the UDG–MWIS primitive of the Rydberg blockade. Variational and analog quantum optimization more broadly is surveyed in [10, 39].

Spectral and force-directed layout. Our layout optimizer draws on classical embedding machinery: the graph Laplacian and its eigenvectors [11, 19], Laplacian eigenmaps [3], force-directed drawing [20], multidimensional scaling [30], and gradient-based optimization [36, 7]. Where the smooth objective is inadequate—e.g. escaping the discrete penalty landscape of false-edge repair—the same role can be played by combinatorial metaheuristics such as simulated annealing [29] or tabu search [22]. These are standard tools; our contribution is not the tools but their re-targeting to a *correctness-neutral* role behind an exact reduction.

Graph decomposition and bounded width. Our scaling layer (Section 9.4) rests on classical structure theory. Chordal (rigid- circuit) graphs always admit clique separators [16], and Tarjan's clique-separator decomposition [44] turns such separators into a divide-and-conquer schedule—exactly the case in which our Stitch rule needs no boundary search. More generally, tree-width and the graph-minors program [41] explain why many NP-hard problems, coloring among them, become tractable on bounded-width instances [1], with linear-time tree-decomposition available for fixed width [5]. The separator search, connected- component analysis, and boundary bookkeeping in our decomposition driver are textbook graph algorithms [14]. Our contribution is to make the width-bounded device budget $(wk)^2 \le N_{\max}$ a *typed* premise (Section 8.4) rather than an after-the-fact analysis.

Type-and-effect systems. The certified-reduction discipline of Section 8 sits in the type-and-effect tradition initiated by Lucassen and Gifford's polymorphic effect systems [32]. Our safety effect—tracking whether a transform preserves or boundedly inflates χ, and accumulating the bound through a monoid—is a small instance of the flow- and resource-analysis algebras developed for process calculi and resource-usage policies [4, 2] and of two-component languages that pair a program with a static analysis of its behavior [15]. What is specific here is that the "effect" is a *combinatorial* guarantee (a chromatic-number bound), gated by the geometric impossibility theorem.

3 Problem Setup and Pipeline Overview

We consider finite simple undirected graphs $G = (V, E)$ with $n = |V|$, $m = |E|$.

Definition 3.1 (Proper k-coloring; chromatic number)**.** A proper k-coloring is a map $c : V \to \{1, \dots, k\}$ with $c(u) \neq c(v)$ for every $\{u, v\} \in E$. The chromatic number $\chi(G)$ is the least k admitting one.

Definition 3.2 (Unit-disk graph)**.** Given a placement $p : V \to \mathbb{R}^2$ and radius $r > 0$, the induced UDG $\mathrm{UDG}(p, r)$ has an edge $\{u, v\}$ iff $\|p(u) - p(v)\|_2 \le r$. A weighted UDG additionally assigns a weight $w : V \to \mathbb{R}_{>0}$.

Definition 3.3 (Independent set; MWIS)**.** An independent set of H is a vertex set with no internal edge. For weights w, the maximum-weight independent set value is $\mathrm{MWIS}(H, w) = \max\{\sum_{v \in S} w(v) : S \text{ independent}\}$. For unit weights this is the independence number $\alpha(H)$.

Hardware primitive. A neutral-atom array with atoms at positions $p(v)$ and blockade radius r realizes $\mathrm{UDG}(p, r)$; under appropriate driving its many-body ground state encodes $\mathrm{MWIS}(\mathrm{UDG}(p, r), w)$, where w is set by local detunings [38, 18]. The device constraints are a minimum atom spacing $d_{\min}$, a maximum blockade radius $r_{\max}$, a finite array area $A_{\max}$, and a maximum atom count $N_{\max}$ [48].

The two-layer pipeline. Figure 1 shows the design. The primary (exact) path reduces coloring to an *abstract* weighted UDG–MWIS instance and decodes the measured independent set back to a coloring. The secondary (heuristic) path is the geometric engine that turns the abstract gadget instance into concrete atom coordinates obeying the hardware constraints, and is also available as a stand-alone — but only chromatic-non-decreasing — coloring backend for comparison.

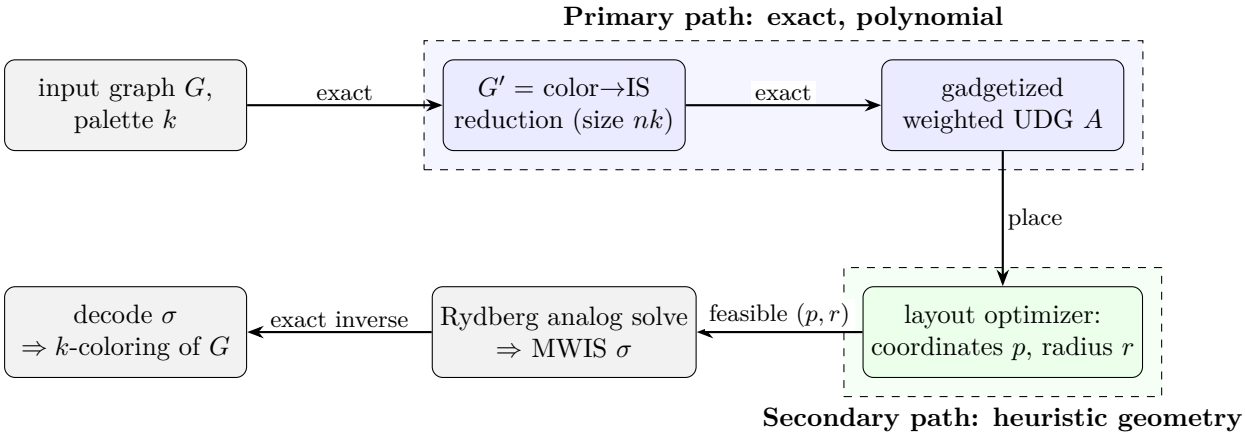

Figure 1: The two-layer pipeline. Correctness lives entirely in the blue (exact) path; the green (heuristic) path determines only hardware feasibility and analog-solve quality.

4 Primary Path: Exact Gadgetized Reduction

The primary path is the composition of two exact reductions:

$$\underbrace{G, k}_{\text{coloring}} \xrightarrow{\text{Stage 1}} \underbrace{G'}_{\text{MIS instance, size } nk} \xrightarrow{\text{Stage 2}} \underbrace{A = (p, r, w)}_{\text{weighted UDG}} \xrightarrow{\text{solve}} \sigma \xrightarrow{\text{decode}} \text{coloring of } G.$$

4.1 Stage 1: Colorability → Independent Set

Definition 4.1 (Color-choice graph)**.** Given $G = (V, E)$ and palette size k, the *color-choice graph* $G' = (V', E')$ has vertex set $V' = V \times \{1, \dots, k\}$, and edge set $E' = E'_{\text{choice}} \cup E'_{\text{conflict}}$ where

$$E'_{\text{choice}} = \{\{(v, i), (v, j)\} : v \in V,\ 1 \le i < j \le k\}, \qquad \text{(a } k\text{-clique per vertex)}$$
$$E'_{\text{conflict}} = \{\{(u, i), (v, i)\} : \{u, v\} \in E,\ 1 \le i \le k\}. \qquad \text{(same-color conflicts)}$$

All vertices carry unit weight. Then $|V'| = nk$ and $|E'| = n\binom{k}{2} + mk$.

The choice-clique enforces "at most one color per vertex"; the conflict edges enforce "adjacent vertices differ in color."

Theorem 4.2 (Exactness of Stage 1)**.** *For every graph G and every $k \geq 1$,*

$$\alpha(G') = n \quad \Longleftrightarrow \quad G \text{ is } k\text{-colorable},$$

and $\alpha(G') \leq n$ always. Moreover, independent sets of G' of size n are in explicit bijection with proper k-colorings of G.

Proof. For each v, the k vertices $\{(v, i)\}_i$ form a clique (E'_{choice}), so any independent set S contains at most one slot per original vertex; hence $|S| \leq n$.

($\Leftarrow$) Let c be a proper k-coloring. Put $S = \{(v, c(v)) : v \in V\}$, so $|S| = n$. S is independent: it has exactly one slot per vertex (no choice edge is internal), and for any $\{u, v\} \in E$ we have $c(u) \neq c(v)$, so no conflict edge $\{(u, c(u)), (v, c(v))\}$ exists — conflict edges only join *equal* color indices. Thus $\alpha(G') \geq n$, and combined with $\alpha(G') \leq n$ we get $\alpha(G') = n$.

($\Rightarrow$) Let S be independent with $|S| = n$. By the choice-clique bound S has exactly one slot per vertex, defining $c(v) = i$ where $(v, i) \in S$. For $\{u, v\} \in E$, if $c(u) = c(v) = i$ then $\{(u, i), (v, i)\} \in E'_{\text{conflict}}$ would be internal to S, contradicting independence. Hence c is proper.

The two maps $c \mapsto S$ and $S \mapsto c$ are mutually inverse, giving the bijection. □

Remark 4.3 (Fixed-k vs. chromatic number). A single value k tests k-colorability. The chromatic number is obtained by a sweep: the least k with $\alpha(G') = n$ equals $\chi(G)$. Binary search over $k \in \{1, \ldots, n\}$ costs $O(\log n)$ solves.

4.2 Stage 2: Independent Set → Unit-Disk MWIS

G' is an abstract graph and is generally not a UDG. Stage 2 embeds it into a weighted UDG that the hardware can realize, using the copy/crossing gadget construction of Nguyen et al. [37], itself building on the Rydberg MWIS encoding of Pichler et al. [38]. We use it as a black box with a precise interface.

Theorem 4.4 (Gadgetized UD-MWIS embedding [37])**.** *There is a polynomial-time procedure* gadgetize *that maps any vertex-weighted graph $G' = (V', E', w')$ to a weighted unit-disk graph $A = (p, r, w)$ on a grid, together with a designated set of* logical *atoms $\ell : V' \hookrightarrow V(A)$ and a constant $C \geq 0$, such that*

$$\text{MWIS}(A, w) = \text{MWIS}(G', w') + C,$$

and every maximum-weight independent set S_A of A restricts on the logical atoms to a maximum-weight independent set $\sigma = \ell^{-1}(S_A \cap \ell(V'))$ of G'. The number of atoms is $|V(A)| = O(|V'|^2)$ in the worst case, governed by the number of edge crossings in the chosen planar layout of G'.

The construction places the $|V'|$ logical atoms along a diagonal and routes each edge of G' as a chain of *copy* gadgets that propagate an atom's occupation; wherever two routed edges must cross, a *crossing* gadget passes both signals through without spurious coupling. Weights are chosen so that the ground-state independent set selects logical atoms exactly according to a maximum independent set of G'. We treat the exact gadget weights as fixed by [37]; choosing or re-deriving a custom gadget set is an engineering fork (Section 11).

4.3 Decoding and End-to-End Correctness

Decoding composes the two inverses. Given a hardware-measured independent set S_A of A: (1) restrict to logical atoms and apply ℓ^{-1} to obtain $\sigma \subseteq V'$ (Stage-2 inverse); (2) if $|\sigma| = n$, read the coloring $c(v) = i$ for the unique $(v, i) \in \sigma$ (Stage-1 inverse). The full chain is the following meta-theorem, proved once for all instances.

Theorem 4.5 (End-to-end exactness)**.** *Let $A = \mathsf{gadgetize}(G')$ for G' the color-choice graph of (G, k). Then:*

(i) $\mathrm{MWIS}(A, w) = n + C$ *if and only if G is k-colorable.*

(ii) Any maximum-weight independent set of A with logical-atom weight n decodes, via ℓ^{-1} followed by the Stage-1 readout, to a proper k-coloring of G.

(iii) The construction and the decoder both run in time polynomial in n and k, and the decoder is an exact inverse of the construction — there is no chromatic-number inflation.

Proof. By Theorem 4.4, $\mathrm{MWIS}(A, w) = \mathrm{MWIS}(G', w') + C = \alpha(G') + C$ (unit weights). By Theorem 4.2, $\alpha(G') = n$ iff G is k-colorable, and $\alpha(G') \leq n$ always; this gives (i). For (ii), a maximum independent set of A restricts (Theorem 4.4) to a maximum independent set σ of G'; when its size is n, the Stage-1 bijection (Theorem 4.2) reads σ off as a proper coloring. Polynomiality in (iii) is immediate: Stage 1 is $O(nk + mk)$ to build, Stage 2 is polynomial by Theorem 4.4, and both readouts are linear in their outputs. Exactness of each inverse was established in the respective theorems, and exactness composes. □

Remark 4.6 (Where uncertainty actually lives). Theorem 4.5 is unconditional: it never assumes the analog solve succeeds. The two real risks are (a) *feasibility* — whether A fits inside $N_{\max}$ atoms and $A_{\max}$ area (Section 4.4), and (b) *analog-solve quality* — whether the Rydberg dynamics reaches the MWIS ground state rather than a low-lying excited state. Neither is a correctness question about the reduction, and a returned independent set is always *verifiable*: $|\sigma| = n$ certifies a valid coloring, $|\sigma| < n$ certifies only that the solver did not find a witness.

4.4 Size and Feasibility Bounds

Proposition 4.7 (Atom budget)**.** *For an instance (G, k) with $|V| = n$, the gadgetized UDG A uses*

$$|V(A)| = O((nk)^2)$$

atoms in the worst case, and at least nk (the logical atoms). The instance is hardware-feasible only if $|V(A)| \leq N_{\max}$ and the layout fits within $A_{\max}$ under spacing $d_{\min}$.

Proof. Stage 1 produces $|V'| = nk$ logical vertices; Stage 2 contributes $O(|V'|^2)$ atoms by Theorem 4.4. The lower bound is the logical-atom count. □

The quadratic blow-up against a few-hundred-atom device [48] is the practical bottleneck, and it is exactly what the layout optimizer of Section 5 attacks. Two preprocessing knobs reduce the effective size before gadgetization:

- **Graph simplification.** Removing vertices of degree $< k$ (they are always colorable last), contracting degree-2 chains, and decomposing on cut vertices and biconnected components all shrink n without changing k-colorability.
- **Crossing minimization.** Because Stage 2's atom count is dominated by edge crossings, a good planar-ish layout of G' (or of G before expansion) directly lowers $|V(A)|$. This is a geometric objective — the province of the heuristics.

4.5 Worked Example: 3-Coloring a Small Graph

Example 4.8. Let G be the 4-cycle C_4 with vertices $v_1v_2v_3v_4$ and $k = 3$. Stage 1 builds G' on $V' = \{(v_a, i) : a \in \{1,2,3,4\},\ i \in \{1,2,3\}\}$, $|V'| = 12$. Each vertex contributes a choice-triangle on its three slots; each edge of C_4 contributes three conflict edges (one per color). A proper 3-coloring such as $c = (1,2,1,2)$ corresponds to the independent set $\{(v_1,1),(v_2,2),(v_3,1),(v_4,2)\}$ of size $4 = n$, and conversely. Since $\chi(C_4) = 2 \le 3$, Stage 1 reports $\alpha(G') = 4$, so G is 3-colorable. Gadgetizing G' yields a weighted UDG with 12 logical atoms plus routing/crossing atoms; measuring its MWIS and applying ℓ^{-1} recovers the size-4 set, whose Stage-1 readout is the coloring $(1,2,1,2)$. No inflation occurs at any step.

5 Secondary Path: Heuristic Geometry and the Layout Optimizer

We now develop the geometric heuristics. We first prove why they cannot serve as the coloring-correctness mechanism, then state the role they actually play.

5.1 Why Geometric Supergraphs Cannot Bound Inflation

Definition 5.1 (Safe supergraph; color inflation)**.** A UDG $H = \mathrm{UDG}(p, r)$ on the same vertex set V as G is a *safe supergraph* if $E(G) \subseteq E(H)$. The *false edges* are $E_{\text{false}} = E(H) \setminus E(G)$, and the *color-inflation ratio* is $\rho = \widehat{\chi}(H)/\widehat{\chi}(G)$, where $\widehat{\chi}$ is a chromatic estimate (e.g. DSATUR).

Proposition 5.2 (Safety $\Rightarrow$ decoder correctness)**.** *If $E(G) \subseteq E(H)$ and κ is a proper coloring of H, then κ restricted to V is a proper coloring of G. Consequently $\chi(G) \le \chi(H)$.*

Proof. Every edge of G is an edge of H, so the endpoints receive distinct colors under κ. The inequality follows since a proper coloring of H is one of G. □

So safe supergraphs never *lower* χ. The problem is the other direction.

Lemma 5.3 (Neighborhood packing in a UDG)**.** *In any $\mathrm{UDG}(p, r)$ and for any vertex v, the open neighborhood $N(v)$ has independence number $\alpha(H[N(v)]) \le 5$.*

Proof. All neighbors of v lie in the closed disk of radius r about $p(v)$. An independent set among them consists of points pairwise at distance $> r$. Partition the disk into six 60° sectors about $p(v)$; two points in the same sector at distance $\le r$ from the center are within $\le r$ of each other (the largest pairwise distance inside a 60° circular sector of radius r is r), hence adjacent. So at most one independent point lies per sector, giving at most 6; a standard refinement excluding the center configuration sharpens the bound to 5 [26]. □

Theorem 5.4 (Supergraph inflation is unbounded)**.** *There is a family of graphs with $\chi = 2$ on which every safe UDG supergraph has $\chi(H) \to \infty$. Concretely, for the star $K_{1,n}$,*

$$\chi(K_{1,n}) = 2, \qquad \chi(H) \ \ge\ \left\lceil \tfrac{n}{5} \right\rceil \quad \textit{for every safe UDG supergraph } H.$$

Proof. $K_{1,n}$ is bipartite, so $\chi = 2$. Let H be a safe UDG supergraph with center c and leaves L, $|L| = n$. Safety forces every edge $\{c, \ell\}$ into H, so all leaves lie within distance r of $p(c)$, i.e. $L \subseteq N(c)$. In any proper coloring of H, each color class is an independent set of H, hence an independent set within $N(c)$, which by Lemma 5.3 has size ≤ 5. Covering the n leaves therefore needs at least $\lceil n/5 \rceil$ color classes, so $\chi(H) \ge \lceil n/5 \rceil$. □

Remark 5.5. Theorem 5.4 is the rigorous reason the supergraph route cannot be "usually safe" for general graphs, and hence cannot be the primary path. It also pinpoints a sufficient structural escape: if the per-vertex neighborhood independence $\sigma(G) = \max_v \alpha(G[N(v)])$ is ≤ 5, the obstruction vanishes, which is the natural precondition under which supergraph safety claims can be scoped.

5.2 The Re-targeted Role: Layout Optimizer for the Gadget Instance

Given Theorem 5.4, the heuristics are *not* used to make $\chi(H) \approx \chi(G)$. Instead, the gadgetized instance A of Section 4.2 fixes the required adjacency *exactly*; the heuristics solve the residual *geometric realization* problem:

> *place the atoms of A in $\mathbb{R}^2$ so that the realized UDG equals the prescribed gadget adjacency, while obeying $d_{\min}$, $r \leq r_{\max}$, area $\leq A_{\max}$, and minimizing atom count / crossings / chain length.*

Crucially, in this role **the heuristics cannot affect correctness**: the gadget structure and weights are fixed by Theorem 4.4, so a poor layout can only fail feasibility (does not fit) or degrade analog-solve quality (harder ground state), never the decoded answer. This is the soundness payoff of Design B.

The optimizer's loss is therefore an *adjacency-realization* loss, not a G-mimicking loss. For the target adjacency E_A of A:

$$L_{\text{layout}}(p, r) = \underbrace{\sum_{\{u,v\} \in E_A} \max(0, d_{uv} - r)^2}_{\text{prescribed edges within } r} + \lambda \sum_{\{u,v\} \notin E_A} \max(0, r + \epsilon - d_{uv})^2 + \mu \sum_{u \neq v} \max(0, d_{\min} - d_{uv})^2, \tag{1}$$

where $d_{uv} = \|p(u) - p(v)\|_2$ and the third term enforces the hardware spacing floor. The seven heuristics below are the components that initialize, drive, repair, and tune this optimization.

5.3 The Seven Heuristics

H1 — Geometric loss minimization. Optimize (1) jointly over coordinates p and radius r by gradient descent [36, 7], $p \leftarrow p - \eta \nabla_p L$, $r \leftarrow r - \eta \nabla_r L$. After convergence apply a **hard feasibility check**: every prescribed edge realized, every non-edge excluded, all pairwise distances $\geq d_{\min}$. If a prescribed edge is missing, set $r \leftarrow \max_{\{u,v\} \in E_A} d_{uv} + \delta$ and rebuild. Cost $O(|V(A)|^2)$ per step.

H2 — Crossing/chain-aware weighting. Edges of A that route through many crossing gadgets are the expensive ones. Weight the loss terms by gadget-chain length so the optimizer prioritizes compacting long routed edges, the analogue of the color-danger weighting of the supergraph setting (DSATUR [8] is used there to score danger; here the score is geometric chain cost). This directly attacks the $O((nk)^2)$ bottleneck of Proposition 4.7.

H3 — Clique/over-packing control. Since $\omega(H) \leq \chi(H)$ and since every UDG clique is geometrically local (contained in some $N[v]$), spurious dense clusters both waste atoms and can corrupt the realized adjacency. Detect them within local neighborhoods in $O(|V(A)|\Delta^2)$ and push offenders apart. The hardware itself bounds clique size:

Proposition 5.6 (Hardware-bounded cliques)**.** *In a physical neutral-atom UDG with blockade radius r and minimum spacing $d_{\min}$, the maximum clique size obeys $\omega_{\max} \leq \lfloor \pi r^2 / (c\, d_{\min}^2) \rfloor$, where c is the planar packing constant ($c \approx 0.9$) [26].*

This gives an a priori cap on local density that depends only on physical parameters [25, 43] and is a useful invariant for the layout.

H4 — Laplacian eigenmap initialization. Initialize p_0 from the Fiedler vector and the next Laplacian eigenvector [19, 11, 3], $p_0(v_i) = (\phi_2(i), \phi_3(i))$. This places adjacent atoms near and well-separated parts far, giving gradient descent a good basin [20]. It is an initializer for H1, not a stand-alone step; cost $O(|V(A)|^2)$ dense, or $O(|V(A)| \cdot t)$ for t eigenvectors via sparse solvers.

H5 — Local repair with a monotone potential. After global optimization, fix residual violations locally by pushing/pulling offending pairs along their connecting line. To guarantee termination, gate every accepted move by strict decrease of an integer-valued potential—a discrete Lyapunov function [33] in the sense that it is bounded below and strictly monovariant along the accepted dynamics—namely

$$\Phi = \sum_{\{u,v\} \in \mathcal{V}} w_{uv},$$

where $\mathcal{V}$ is the current set of adjacency violations and $w_{uv} \geq 1$ are integer weights. Since $\Phi \geq 0$ is integer and drops by ≥ 1 per accepted step, at most $\Phi_0 \leq |V(A)|^2 \max w$ steps occur — a genuine polynomial bound (unlike a vague "until convergence").

H6 — Radius sweep and Pareto selection. With p fixed, sort all pairwise distances and sweep r. The *minimum feasible radius* $r^\star = \max_{\{u,v\} \in E_A} d_{uv}$ is the smallest r realizing all prescribed edges; for $r < r^\star$ the layout is infeasible. Over $r \in [r^\star, r_{\max}]$ select a Pareto point minimizing (realized non-edges, atom area, r) subject to $r \leq r_{\max}$ [35]. Cost $O(|V(A)|^2 \log |V(A)|)$ to sort plus $O(|V(A)|^2)$ per evaluated radius.

H7 — Inflation/quality as evaluation, not objective. Discrete quality measures (number of realized non-edges, decoded solution gap) are piecewise-constant in (p, r) and have no useful gradient, so they must be used to *evaluate* and *select* layouts (e.g. at the H6 sweep) rather than as differentiable objectives. For the optional stand-alone supergraph backend, the same caveat applies to ρ.

5.4 Corrected Exactly-Realizable Families

For the optional stand-alone supergraph backend it is useful to know families that embed with *zero* false edges ($\rho = 1$). The naive folklore claims require correction.

Proposition 5.7 (Caterpillars, not all trees)**.** *Not every tree is a UDG: the star $K_{1,6}$ is a tree that is not a unit-disk graph (Theorem 5.4 with $n = 6$ forces $\chi(H) \geq 2$ false structure, and indeed $K_{1,6}$ has no UDG realization). However, every* caterpillar *(a tree whose non-leaf vertices form a path) is a proper-interval graph and hence realizes as a 1D UDG with $\rho = 1$.*

Proposition 5.8 (Unit-interval, not all interval graphs)**.** *The 1D unit-disk graphs are exactly the* unit-interval *(equivalently proper-interval) graphs [40, 23]. Not every interval graph qualifies: the claw $K_{1,3}$ is an interval graph but not unit-interval. The claim "interval graphs are 1D UDGs" must be restricted to unit-interval graphs.*

Remark 5.9. The earlier blanket assertions "all trees achieve $\rho = 1$," "all interval graphs are 1D UDGs," and an unconditional "bipartite UDGs achieve $\rho = 1$" are false or vacuous and are replaced here by Propositions 5.7–5.8; the bipartite claim is dropped as it presupposes the embedding it purports to construct.

6 Integrated Pipeline

Algorithm 1 states the end-to-end compiler. Steps 1–2 are the exact primary path; steps 3–7 are the heuristic layout of the resulting gadget instance; step 8 solves and decodes.

Input: graph $G = (V, E)$, palette size k (or sweep for χ), hardware parameters $(d_{\min}, r_{\max}, A_{\max}, N_{\max})$.
Output: proper k-coloring of G, or a feasibility/solve-failure certificate.

1. **Stage 1 (exact):** build color-choice graph G'. *// $O(nk + mk)$; Thm. 4.2*
2. **Stage 2 (exact):** $A, \ell, C \leftarrow \mathsf{gadgetize}(G')$. *// poly; Thm. 4.4; check $|V(A)| \le N_{\max}$*
3. **H4:** $p_0 \leftarrow \mathrm{LaplacianEigenmap}(A)$. *// initialize layout*
4. **H1–H2:** minimize $L_{\text{layout}}(p, r)$ for a fixed $T = \mathrm{poly}(|V(A)|)$ budget. *// Eq. (1)*
5. **Feasibility check:** all prescribed edges realized, spacing $\ge d_{\min}$; if not, raise r or rebuild.
6. **H3:** break over-packed clusters. *// Φ-gated*
7. **H5:** local repair of residual violations. *// Φ-gated, terminates*
8. **H6:** radius sweep; pick Pareto $(p, r^\star)$ with $r^\star \le r_{\max}$, area $\le A_{\max}$.
9. **Solve:** run Rydberg MWIS on A at $(p, r^\star)$; measure S_A.
10. **Decode:** $\sigma \leftarrow \ell^{-1}(S_A)$; if $|\sigma| = n$ return the Stage-1 coloring readout, else report solve failure. *// Thm. 4.5*

Algorithm 1: Certified coloring-to-neutral-atom compiler

7 Complexity Analysis

Let $N = |V(A)| = O((nk)^2)$ be the atom count. Table 1 summarizes the per-stage cost. Every stage is polynomial; the exact path is low-degree polynomial, and the dominant cost is the layout optimization over N atoms.

Stage	Cost	Type	Notes
Stage 1 (color→IS)	$O(nk + mk)$	exact	build G'
Stage 2 (gadgetize)	$\mathrm{poly}(nk)$, $N = O((nk)^2)$	exact	Thm. 4.4
H4 Laplacian init	$O(N^2)$ (one-time)	heuristic	sparse: $O(Nt)$
H1/H2 optimization	$O(T\,N^2)$	heuristic	$T = \mathrm{poly}(N)$ budget
H3 clique control	$O(N\Delta^2)$ per pass	heuristic	local
H5 local repair	$\le \Phi_0$ steps, $\Phi_0 = O(N^2 \max w)$	heuristic	Φ-gated, terminates
H6 radius sweep	$O(N^2 \log N)$	heuristic	$+\ O(N^2)$/radius
Decode	$O(N)$	exact	ℓ^{-1} + readout

Table 1: Per-stage complexity. The NP-hardness of UDG *recognition* [9] is never incurred: we never decide whether G is a UDG, only realize a prescribed gadget adjacency.

Proposition 7.1 (Polynomial compilation)**.** *With a fixed iteration budget $T = \mathrm{poly}(N)$, Algorithm 1 runs in time polynomial in n and k for everything except the analog solve, and emits a hardware instance of $O((nk)^2)$ atoms together with an exact decoder.*

Proof. Sum the rows of Table 1; each is polynomial in $N = O((nk)^2)$ and $T = \mathrm{poly}(N)$. Exactness of the emitted decoder is Theorem 4.5. □

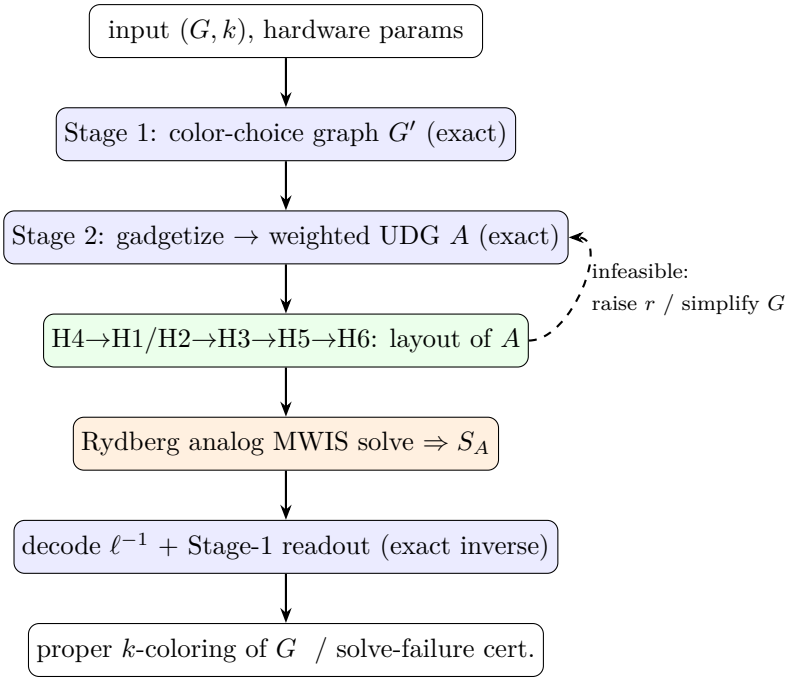

Figure 2: Data flow of Algorithm 1. Blue = exact, green = heuristic layout, orange = analog hardware. Only feasibility, never correctness, flows back.

8 The Coloring Language and Its Certified-Reduction Type System

The pipeline of the preceding sections is not a fixed script; it is the denotation of programs in a small *coloring-specification language*, and its correctness is enforced by a *type system* whose types are not data shapes but *problem reductions*. This section gives the surface language (Section 8.1), the problem-type sorts and the single certified- reduction judgment (Section 8.2), the per-stage typing rules and the soundness theorem they compose to (Section 8.3), and the three refinements that capture certificate strength, the supergraph safety effect, and decomposition (Section 8.4). The slogan is: *base types say a program is well-formed; problem-types say a* ***reduction*** *is faithful.* We build only the second; ordinary scope/arity/base-type checking is delegated to a standard typed core. Sections 9.1–9.2 then report the axiom-free Coq mechanization of this calculus and the Haskell checker that implements it.

8.1 The Surface Language

A program is a sequence of declarations: a `graph`, a `palette` of colors, optional `restrictions` (precolorings, allow/forbid lists, equality constraints, color budgets), a `transform` into the unit-disk world, an `instance` that ties these together, and a `compute` query. Figure 3 gives the core grammar; Figure 4 a complete program. The two transform *modes* are exactly the paper's two notions of transformation: `toUDG(exact)` demands a faithful unit-disk realization, while `toUDG(preserve` P`)` only promises to preserve a property P (e.g. `chromatic`, `kColorable(`k`)`)—the supergraph path, which the type system gates (Section 8.4).

```
Decl       ::= "graph" Id "=" GraphExpr | "palette" Id "=" PaletteExpr
             | "restrictions" Id "=" "{" { Restriction ";" } "}"
             | "transform" Id "=" TransformExpr
             | "instance" Id "=" InstanceExpr | "compute" Id "=" QueryExpr
InstanceExpr ::= "color" GraphRef [ "using" PaletteRef ]
                 [ "subject-to" RestrRef ] [ "via" TransformRef ]
TransformExpr ::= "toUDG" "(" TransformMode [ "," TransformOptions ] ")"
                | "compose" "[" TransformRefList "]"
TransformMode ::= "exact" | "preserve" Property
Property      ::= "kColorable" "(" Nat ")" | "chromatic" | "bipartite"
                | "paletteFeasible"
Restriction   ::= "precolor" Vertex ":" Color | "allow" Vertex ":" ColorList
                | "forbid" Vertex ":" ColorList | "sameColor" "(" V "," V ")"
                | "differentColor" "(" V "," V ")" | "useAtMost" Nat
                | "useExactly" Nat | "distinctOn" VertexList
QueryExpr     ::= "chromaticNumber" "(" GraphRef ")"
```

Figure 3: Core grammar of the coloring-specification language (elided: `GraphExpr`/`PaletteExpr` literals and lexical classes). The full BNF, AST, and transform options appear in the companion grammar document.

```
graph gInput = { vertices=[1,2,3,4,5], edges=[(1,2),(2,3),(3,1),(1,4),(3,5)] }
palette pNamed = colors [red, blue, green]
restrictions rMain = { precolor 1 : red; differentColor(1,2); useAtMost 3 }
transform tExact    = toUDG(exact, radius=1.0)
transform tPreserve = toUDG(preserve chromatic, keep=[1,2,3], trackOrigin=true)
instance inst1 = color gInput using pNamed subject-to rMain via tPreserve
compute  chiInput = chromaticNumber(gInput)
```

Figure 4: A well-typed program. `inst1` colors `gInput` with a 3-palette under `rMain`, compiled through the property-preserving transform; `chiInput` queries the chromatic number. This program is among those the implementation type-checks (Section 9.2).

8.2 Problem-Types and the Certified-Reduction Judgment

A *problem-type* (sort) names a class of decision/optimization instances together with the indices needed to state preservation:

$A, B \;::=$	$\mathsf{Col}\langle n, k\rangle$	k-coloring instance, $n = \lvert V\rvert$
	$\mathsf{Col}\langle n, k \mid \beta\colon B\rangle$	boundary-precolored (β proper on $B \subseteq V$)
	$\mathsf{IS}\langle N\rangle\{P\}$	independent-set instance, N nodes, refinement P
	$\mathsf{WIS}\langle N,\, C\rangle$	weighted IS, N nodes, additive offset C
	$\mathsf{UDG}\langle N,\, r,\, d\rangle$	*exactly* realized unit-disk inst. (radius r, spacing d)
	$\widetilde{\mathsf{UDG}}\langle N,\, r,\, d \mid f\rangle$	*approximately* realized, f residual false edges
	$\mathsf{Dev}\langle N\rangle \quad (N \le N_{\max})$	device-feasible instance

Each sort A carries a solution domain $\mathrm{Sol}(A)$ and an optimum $\mathrm{opt}(A)$. The refinement on IS is where Stage-1 exactness lives; the canonical one is $P_{\mathrm{s1}} \equiv (\alpha = n \iff G \text{ is } k\text{-colorable}) \wedge \alpha \le n$. The two unit-disk sorts encode the paper's central honesty: UDG is the placement whose realized graph equals the prescribed gadget adjacency *exactly*, whereas a layout that leaves false edges is genuinely a *different* graph and so gets the distinct sort $\widetilde{\mathsf{UDG}}$—which, crucially, **is not a legal source for the exact decoder**. The type is what forbids reading an answer off a corrupted layout.

Definition 8.1 (Decoder, total on optima)**.** A *decoder* from B to A is a partial map $\delta : \mathrm{Sol}(B) \rightharpoonup \mathrm{Sol}(A)$. It is *total on optima*, $\mathsf{tot\text{-}opt}(\delta)$, if it is defined on every value-opt(B) solution and maps it to a value-opt(A) solution of A.

Definition 8.2 (Certified reduction)**.** The judgment $\vdash \tau : A \overset{\delta}{\rightsquigarrow} B$ asserts that τ takes an instance of A to an instance of B, that δ is a decoder $B \rightharpoonup A$ with $\mathsf{tot\text{-}opt}(\delta)$, and that any optimum $s^\star$ of B yields the answer to A as $\delta(s^\star)$. Operationally: *solve B on the device, apply δ, obtain the answer to A.*

This single arrow is the backbone: stage rules introduce one arrow each, composition chains them, and refinements add indices (grade, effect, budget).

8.3 Per-Stage Typing Rules and Soundness

Premises above each line are *proof obligations*, discharged once and for all by the meta-theorems of Sections 4 (and, in the implementation, by the brute-force oracle on small instances). Stage 1 is the color-choice reduction; Stage 2 the gadget embedding with its static offset C; Stage 3 branches on geometric realizability; Fit crosses onto hardware.

$$\frac{n = |V(G)| \quad k \geq 1}{\vdash \mathsf{colorChoice}_k : \mathsf{Col}\langle n, k\rangle \overset{\mathsf{decodeColoring}}{\rightsquigarrow} \mathsf{IS}\langle nk\rangle\{P_{\mathrm{s1}}\}} \ \text{S1}$$

$$\frac{C = 2\,|E(G')|}{\vdash \mathsf{gadgetize} : \mathsf{IS}\langle N\rangle\{P\} \overset{\mathsf{decodeLogical}}{\rightsquigarrow} \mathsf{WIS}\langle N{+}C,\ C\rangle} \ \text{S2}$$

$$\frac{\mathsf{realizes}(p, r) \quad \mathsf{spacing}(p) \geq d}{\vdash \mathsf{layout}(p, r) : \mathsf{WIS}\langle N, C\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{UDG}\langle N, r, d\rangle} \ \text{S3-exact}$$

$$\frac{\mathsf{miss}(p, r) = \varnothing \quad f = |\mathsf{false}(p, r)| > 0}{\vdash \mathsf{layout}(p, r) : \mathsf{WIS}\langle N, C\rangle \rightsquigarrow \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle} \ \text{S3-approx}$$

$$\frac{N \leq N_{\max}}{\vdash \mathsf{emit} : \mathsf{UDG}\langle N, r, d\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle} \ \text{Fit} \qquad \frac{\vdash \tau_1 : A \overset{\delta_1}{\rightsquigarrow} B \quad \vdash \tau_2 : B \overset{\delta_2}{\rightsquigarrow} C}{\vdash \tau_2 \circ \tau_1 : A \overset{\delta_1 \circ \delta_2}{\rightsquigarrow} C} \ \text{Comp}$$

Rule S3-exact carries the identity decoder (geometry never relabels a solution) and lands in UDG; S3-approx carries *no* certified decoder and lands in $\widetilde{\mathsf{UDG}}$. The only exit from $\widetilde{\mathsf{UDG}}$ is a metric Repair step ($\widetilde{\mathsf{UDG}} \overset{\mathsf{decodeRoute}}{\rightsquigarrow} \mathsf{WIS}$) that re-routes false edges through the copy/crossing gadgets, after which S3-exact may be retried—the type-level form of the deferred engineering fork.

Theorem 8.3 (Pipeline soundness)**.** *If $\vdash \Pi : \mathsf{Col}\langle n, k\rangle \overset{\delta}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle$ is derivable (so $N \leq N_{\max}$ and every layer realized* exactly*), then for any device measurement m of value* opt($\mathsf{Dev}\langle N\rangle$)*, the decoded $\delta(m)$ is a proper k-coloring of G when one exists, and otherwise the pipeline reports "not k-colorable" soundly.*

Proof sketch. Induction on the derivation via Comp: each arrow is answer-preserving with a $\mathsf{tot\text{-}opt}$ decoder, and $\mathsf{tot\text{-}opt}$ decoders compose, so $\delta = \delta_{\mathrm{s1}} \circ \delta_{\mathrm{s2}} \circ \mathrm{id} \circ \mathrm{id}$ maps an optimum bitstring back through WIS (strip C) and IS (one slot per vertex) to a coloring; the refinement P_{s1} supplies the iff. The only non-identity geometric step, S3-exact, has premise $\mathsf{realizes}(p, r)$ guaranteeing the realized graph *is* A, so their optima coincide. This is exactly the meta-theorem of Theorem 4.5, here read off the derivation. □ □

Corollary 8.4 (Ill-ordered pipelines do not type)**.** *The three classic stage-ordering errors are rejected structurally, with no side-analysis: S2 (*gadgetize before colorChoice*) expects* IS *but is given* Col*; Fit (*emit before layout*) expects* UDG *but is given* WIS*; and* reading an answer off a false-edged layout *has no* tot-opt *decoder out of* $\widetilde{\mathsf{UDG}}$ *(only Repair, which goes* back *to* WIS*). The device is reachable* ***only through*** UDG*, so a measured bitstring is connected to a correct coloring by a chain of* tot-opt *decoders—or it is not connected at all, and the checker says so.*

8.4 Three Refinements

Refinement A: certificate grade. A grade $g \in \{\mathsf{w}, \chi\}$ on coloring conclusions, with $\mathsf{w} \sqsubseteq \chi$, distinguishes a cheap witness-relative certificate ($\mathsf{Col}^{\mathsf{w}}$: a carried coloring κ proves $\chi(G) \le |\kappa|$ in poly time, the default) from a proof-only exact one (Col^{χ}: $\chi(G) = k$ as a discharged lower bound). All stage rules are grade-polymorphic and preserve the grade; only an explicit prove_{χ} step lifts $\mathsf{w} \Rightarrow \chi$. Theorem 8.3 already holds at grade w: *soundness of the compilation does not require knowing the true chromatic number.*

Refinement B: the safety effect. The supergraph path rewrites G on the *same* vertices by adding edges (decoded by restriction), which may inflate χ, so such a transform carries a *safety effect* $s \in \{\mathsf{safe}, \mathsf{asafe}(c)\}$: safe preserves χ exactly; $\mathsf{asafe}(c)$ never lowers χ and bounds its increase by c. Effects form a commutative monoid that accumulates the bound, $\mathsf{safe} \oplus \mathsf{safe} = \mathsf{safe}$, $\mathsf{safe} \oplus \mathsf{asafe}(c) = \mathsf{asafe}(c)$, $\mathsf{asafe}(c) \oplus \mathsf{asafe}(c') = \mathsf{asafe}(c{+}c')$, and composition combines decoders and effects together. Crucially, a $\mathsf{safe}/\mathsf{asafe}(c)$ introduction is derivable *only* under the neighborhood bound $\sigma(G) = \max_v \alpha(G[N(v)]) \le 5$; otherwise the sole available type is unsafe (no χ certificate). This side-condition is precisely the impossibility obstruction of Theorem 5.4 ($\chi(K_{1,n}) = 2$ versus $\chi(H) \ge \lceil n/5 \rceil$), refusing to type *as the type system.* Because $\oplus$ sums the bumps while the carrier graph differs between orders, two orderings of the same transforms are type-equal only if their accumulated bounds and side-conditions both match—so the type *records* transform-order sensitivity as a checkable property rather than a conjecture.

Refinement C: decomposition (split/stitch). Decomposition makes the discipline substructural: pieces are resources and a shared separator is a value the pieces must agree on. A balanced separator S with pieces $G_i = G[S \cup C_i]$ of width $w = \max_i |G_i|$ splits the goal into subgoals, each compiled independently:

$$\frac{S \text{ separates } G \quad w = \max_i |G_i| \quad (wk)^2 \le N_{\max}}{\vdash \mathsf{split}_S : \mathsf{Col}\langle n, k\rangle \Longrightarrow \{\, \mathsf{Col}\langle |G_i|, k \mid \beta_i{:}S\rangle \,\}_{i=1}^{p}} \textsc{Split} \qquad \frac{\forall i,\ \kappa_i \text{ proper}; \quad \kappa_i|_S = \kappa_j|_S}{\vdash \mathsf{stitch} : \{\kappa_i\}_i \overset{\bigcup}{\rightsquigarrow} \kappa \text{ on } G} \textsc{Stitch}$$

The typed budget $(wk)^2 \le N_{\max}$ makes *peak device size depend on the separator width w, not on n*—register spilling, for atoms. For a clique separator the agreement premise is discharged for free by a color-relabeling coercion; for a general separator it is a bounded search over the $k^{|S|}$ boundary table. This rule is exactly what the empirical decomposition of Section 9.4 executes: the $\le$ 13-atom pieces against nk up to 192 are instances of Split meeting its budget premise, and the DIMACS Mycielski non-colorability proof is the case where every boundary β fails.

Theorem 8.5 (Decomposed soundness)**.** *If Split produces pieces each compiled by a derivation of Theorem 8.3, and Stitch applies, then $\kappa = \bigcup_i \kappa_i$ is a proper k-coloring of G, using peak device size $O((wk)^2) \le N_{\max}$.*

Section 9.1 mechanizes the core of this calculus—sorts, the tot-opt arrow, rules S1/S2/Fit/Comp, the soundness theorem, the ordering corollary, and the effect monoid—axiom-free in Coq.

9 Implementation, Mechanized Proofs, and Empirical Validation

The design above is backed by a working implementation in three artifacts: an axiom-free *Coq/Rocq* mechanization of the correctness theorems (Section 9.1); a prototype *typed front end* in Haskell that parses, type-checks, elaborates, and emits device instances (Section 9.2); and a *Rydberg adiabatic simulator* that solves the compiled instances and decodes colorings, validated against a brute-force oracle (Sections 9.3–9.5). Every numeric result in this section is reproduced by the scripts in the accompanying code, and every theorem cited as "machine-checked" is closed under the global context (no axioms, no `Admitted`).

9.1 Mechanized Correctness in Coq

We formalize the certified-reduction calculus of Section 8 in the Rocq Prover (Coq 9.0.0) across five modules and *no* external libraries. The problem-type sorts of Section 8.2, the tot-opt decoder, and the rules S1, S2, Fit, and Comp become Coq definitions; the soundness Theorem 8.3 and the ordering Corollary 8.4 become machine-checked theorems. A *problem type* bundles a solution set with an optimality predicate; a *certified reduction* $\mathsf{CRed}\, A\, B$ carries an executable decoder $B \rightharpoonup A$ together with a *total-on-optima* certificate guaranteeing that every target optimum decodes to a source optimum. Reductions compose, the certificate composes, and the generic soundness lemma falls out once and is reused at every stage. Crucially, the Stage-1 "choice clique" (at most one color slot per vertex) is encoded *exactly* by an `option`-valued selection, which eliminates all pigeonhole counting and makes the exactness theorem a clean equivalence rather than an inequality chase. Table 2 lists the head theorems; all are verified axiom-free.

Theorem (module)	Statement	Role
`cred_sound` (Problem)	target optimum $\Rightarrow$ decoded source optimum	soundness engine
`stage1_exact` (Stages)	$\alpha(G') = n \iff k$-colorable	Thm. 4.2
`gadget_mwis` (Stages)	MWIS $= \alpha + C$	offset invariant
`pipeline_sound` (Pipeline)	optimal device readout $\Rightarrow$ proper k-coloring	Thm. 4.5
`pipeline_decision` (Pipeline)	device optimum exists $\iff$ colorable	decision form
`dev_only_via_udg` (Pipeline)	device reachable only through exact UDG sort	ordering
`bound_eplus` (Effect)	$\mathrm{bound}(a \oplus b) = \mathrm{bound}\, a + \mathrm{bound}\, b$	χ-bounds add

Table 2: Machine-checked head theorems (Rocq 9.0.0, axiom-free). The three ill-ordered pipelines—gadgetizing before the choice graph, emitting before layout, exiting the lossy false-edged layout—have *no* derivation in the step relation (lemmas `no_gadget_before_choice`, `no_emit_before_layout`, `no_exit_from_udga`), making the legal stage order a corollary of typing rather than a convention. The effect algebra $\{\mathsf{safe}, \mathsf{asafe}\, c\}$ is proved a commutative monoid under $\oplus$.

9.2 The Typed Front End

A companion Haskell package (`udg-color`, GHC 9.12) implements the surface pipeline end to end: a `parser` and AST for the coloring-specification language; a `type-and-effect checker` that enforces provenance, stage ordering, and the feasibility refinement $|V(A)| \le N_{\max}$; an `elaborator` that lowers a well-typed program through Stage 1 and gadgetization; and an `emitter` that writes the device instance consumed by the simulator. The checker realizes the static discipline of Section 8 (mechanized in Section 9.1): it accepts the worked program of Figure 4, rejects a stand-alone supergraph transform whose neighborhood bound $\sigma(G) > 5$

would void the safety effect of Section 8.4, and raises the catalogued ill-typing errors. That catalogue (Table 3) is the negative space of the type system—each entry is a program the checker must *reject*, and the test suite confirms it does. Two test suites pass: a language suite (4/4 acceptance/rejection checks, including the $\sigma > 5$ rejection and the error codes of Table 3) and a property suite that runs 12,226 checks against the brute-force oracle (Stage-1 sizes, decoder round-trips, gadget offsets) and renders ten visualized examples—12,230 checks in total.

#	Ill-typed specification	Violated rule/index	Caught by
1	palette where a graph is expected	sort/kind mismatch	core types
2	restriction names a vertex $\notin V$	$\mathsf{Col}\langle n, k\rangle$ provenance	env. lookup
3	edge endpoint $\notin V$	graph well-formedness	graph check
4	color $\notin$ palette	palette refinement	palette check
5	`useExactly`$>$ |palette|	palette-size refinement	arithmetic
6	restriction set reused on wrong graph	provenance (instance id)	S1 source
7	`keep`/`drop` vertex $\notin V$	transform domain	domain check
8	overlapping `keep`/`drop`	transform well-formedness	set disjointness
9	`exact` transform that drops vertices	S3-EXACT (no elimination)	mode check
10	weak transform used as a stronger guarantee	grade $\mathsf{w} \not\sqsupseteq \chi$	guarantee lattice
11	color used in a vertex position	value sort	core types
12	UDG literal whose points/edges disagree	UDG realizability	literal check

Table 3: The ill-typed catalogue the checker rejects. Errors 6 and 10 are the provenance and guarantee-grade violations; the stage-ordering errors of Corollary 8.4 (gadgetize-before-choice, emit-before-layout, decode-from-$\widetilde{\mathsf{UDG}}$) are rejected structurally by the absence of an applicable rule. The language test suite exercises a representative subset and confirms each is raised.

9.3 Validating the Reduction by Rydberg Simulation

To confirm that the compiled instances are not merely well-typed but actually *solve*, we built a faithful neutral-atom simulator. It evolves the time-dependent Rydberg Hamiltonian under the QuEra-style adiabatic protocol (ramp the Rabi drive Ω up then down while sweeping the detuning Δ from negative to positive), with the blockade placed on the edges of the color-choice graph G'; reading out the most-populated basis states and post-selecting genuine independent sets reproduces exactly what hardware does over many shots. We compare the recovered |MIS| against a brute-force oracle that enumerates the $(k{+}1)^n$ partial colorings.

Across ten named benchmark graphs, each run at $k = \chi$ (colorable, oracle $\alpha(G') = n$) and $k = \chi - 1$ (not colorable, $\alpha(G') < n$), **all 18 simulated instances match the oracle exactly**, and every colorable case decodes to a verified-proper coloring (Table 4). The simulator cost grows as the 2^{nk} state vector demands (Figure 5): instances stay sub-second up to ≈ 12 slots and reach $\approx 55\,\mathrm{s}$ at the 16-slot wall (2^{16} amplitudes), at which point Petersen ($nk = 30$) is correctly skipped. This wall is a *classical-simulator* limit, not the device budget $N_{\max}$ nor anything fundamental to the reduction—and Section 9.4 breaks it.

9.4 Defeating the Simulation Wall by Certified Decomposition

The 2^{nk} wall is dissolved by a *separator decomposition* of the coloring instance, the classical mirror of the type system's split/stitch discipline. Given a balanced vertex separator S of G, we precolor S, solve each component + boundary independently on a *small* device instance, and stitch the pieces (they agree on S by construction); when no separator exists we pin one free vertex and branch over its k colors. Because each device solve sees only a component's free vertices plus its (small) boundary, the peak simulated piece is governed by the separator width, *independent of* n. The recursion is exact: pinning a vertex never changes colorability,

Graph	n	m	χ	slots nk	oracle = sim	proper
P_4 (path)	4	3	2	8	$4 = 4$	✓
C_4 (cycle)	4	4	2	8	$4 = 4$	✓
C_5 (cycle)	5	5	3	15	$5 = 5$	✓
K_3 (triangle)	3	3	3	9	$3 = 3$	✓
K_4 (complete)	4	6	4	16	$4 = 4$	✓
$K_{2,3}$	5	6	2	10	$5 = 5$	✓
$K_{1,4}$ (star)	5	4	2	10	$5 = 5$	✓
Bull	5	5	3	15	$5 = 5$	✓
W_5 (wheel)	5	8	3	15	$5 = 5$	✓
Petersen	10	15	3	30	skipped ($> N_{\max}^{\text{sim}} = 16$)	

Table 4: Reduction validated by simulation. Each graph is run at $k = \chi$ and $k = \chi - 1$; all 18 simulated instances reproduce the brute-force oracle's $\alpha(G')$ exactly, and every $k = \chi$ case decodes to a proper coloring. The $k = \chi - 1$ rows (omitted for space) likewise match, correctly reporting non-colorability. Petersen exceeds the 16-slot classical-simulation wall.

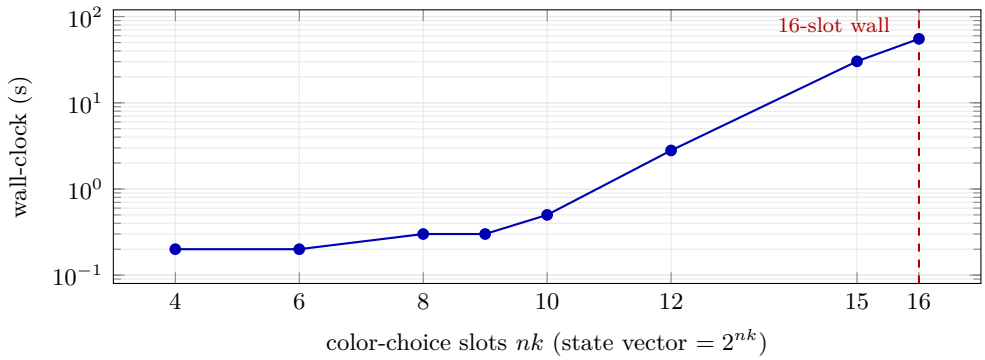

Figure 5: Classical-simulation cost is exponential in the number of slots nk (semilog axis): sub-second to ≈ 12 slots, ≈ 55 s at the 2^{16} wall. The next subsection removes this wall without enlarging any single solve.

and exhausting the boundary table constitutes a proof of non-colorability when every branch fails.

Table 5 demonstrates this on graphs whose whole-instance nk ranges up to 192—a 2^{192} monolithic state vector, utterly impossible—while **every Rydberg solve stays at ≤ 13 atoms** and the decoded coloring is verified proper (or, for the odd cycle C_{21} at $k = 2$, correctly reported non-colorable). Figure 6 contrasts the monolithic nk with the largest piece actually simulated; the gap is the contribution of decomposition.

9.5 The DIMACS Mycielski Benchmark

For a recognized benchmark we turn to the *Mycielskian* family `myciel3`–`myciel6` from the DIMACS coloring suite: triangle-free graphs whose chromatic number nonetheless grows without bound, the textbook "hard-to-color" instances on which clique and greedy bounds are useless. We run each instance in *both directions*, which together **certify the chromatic number exactly**: the feasible direction $k = \chi$ must find a proper coloring, and the infeasible direction $k = \chi - 1$ must *prove* that none exists by exhausting the boundary table. The latter is the decisive correctness test—it is the expensive, backtracking-heavy direction that a mere coloring-finder never exercises.

Table 6 reports the result: the pipeline certifies $\chi = 4$ for `myciel3` and finds a 5-coloring of `myciel4` ($\chi = 5$), with every device solve held to ≤ 11 atoms against monolithic instances of 2^{44}

Graph	n	m	k	whole nk	max piece	device calls	verdict
C_{24} (cycle)	24	24	8	192	11	8	proper (5 colors)
Ladder 2×10	20	28	8	160	11	10	proper (4 colors)
Dodecahedral	20	30	8	160	13	7	proper (7 colors)
Star $K_{1,12}$	13	12	10	130	11	12	proper (2 colors)
Petersen	10	15	8	80	13	3	proper (6 colors)
Desargues	20	30	2	40	11	7	proper (bipartite)
Dodecahedral	20	30	3	60	10	25	proper ($\chi = 3$)
C_{30} (cycle)	30	30	3	90	11	8	proper
C_{21} (odd)	21	21	2	42	12	17	**not 2-colorable**

Table 5: Certified decomposition. The whole color-choice instance (nk slots) is classically unsimulable, yet the largest device piece is always ≤ 13 atoms. All verdicts are correct: proper colorings where colorable, a sound non-colorability proof for the odd cycle.

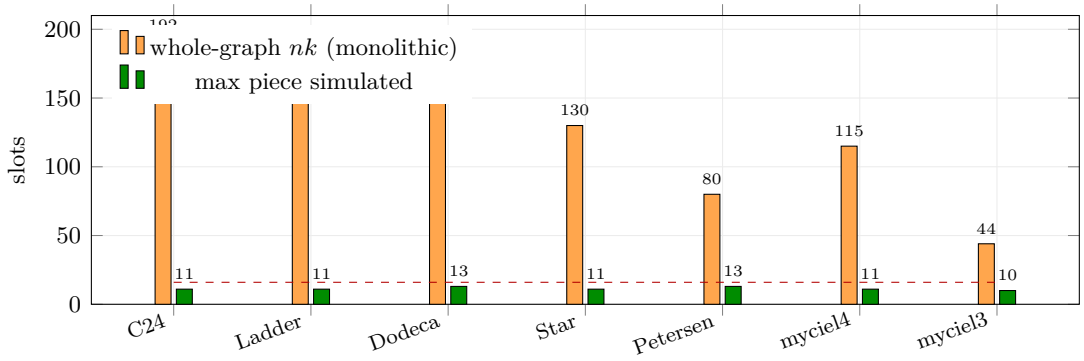

Figure 6: Whole-instance nk (orange, would require a 2^{nk} vector) versus the largest piece actually simulated (green) under decomposition. The dashed line is the 16-slot monolithic wall; every device solve stays comfortably below it while nk reaches 192.

and 2^{115}. The non-3-colorability proof for `myciel3` costs 158 device calls and 82 s—real work, soundly discharged. The honest ceiling is `myciel4`: `myciel5` ($n = 47$) and dense instances such as the queen graphs have large separators that the decomposition cannot keep small, and they exceed our reach; this is the same size/structure wall discussed in Section 10, made concrete.

Instance	n	m	χ	k	direction	whole nk	max piece	verdict
`myciel3`	11	20	4	4	feasible	44	10	proper 4-coloring
`myciel3`	11	20	4	3	infeasible	33	11	**proves no 3-coloring**
`myciel4`	23	71	5	5	feasible	115	11	proper 5-coloring

Table 6: DIMACS Mycielski benchmark, run through the full decomposition+Rydberg pipeline. Both directions on `myciel3` certify $\chi = 4$ exactly; the infeasible row is a sound, exhaustive non-colorability proof (158 device calls, 82 s). All verdicts agree with the known chromatic numbers, and no single device solve exceeds 11 atoms.

9.6 The Genuine Analog Emulator

Finally, to close the loop on the analog physics rather than an abstract edge-blockade Hamiltonian, the simulator can dispatch to the genuine `bloqade-analog` emulator on real atom coordinates, where the blockade arises from the geometric C_6/r^6 Rydberg interaction. On a five-atom unit-disk "bowtie" (two triangles sharing an apex, MIS $= \{0, 4\}$ of size 2), the geomet-

ric emulator and the abstract engine agree on the maximum independent set, confirming that the adiabatic protocol used throughout is the literal neutral-atom dynamics and not an idealization. The color-choice graphs G' are themselves generally non-unit-disk, which is precisely why the exact gadgetization of Section 4 is needed to render them on hardware; the abstract engine faithfully simulates that logical array on the CPU for the instance sizes our pipeline produces.

10 Discussion and Limitations

What is and isn't guaranteed. The reduction is exact and verifiable: a decoded set of size n is a certified coloring. What is *not* guaranteed is that the analog device reaches the MWIS ground state, or that a large instance fits the array. These are physics and capacity questions, cleanly separated from correctness by Theorem 4.5.

The size wall. The $O((nk)^2)$ atom budget is the binding constraint against present devices of a few hundred atoms [48], and our evaluation makes it concrete: the classical simulator hits a 2^{nk} wall near 16 slots (Figure 5). The leverage is preprocessing (graph simplification, crossing minimization), the layout optimizer, and—demonstrated in Section 9.4—separator decomposition, which keeps every device solve to ≤ 13 atoms while the whole-instance nk reaches 192. The remaining limitation is structural: decomposition only helps when the instance has small balanced separators. Dense, large-separator instances (the DIMACS queen graphs, `myciel5` and beyond) stay out of reach, and our certified runs accordingly top out at `myciel4`. Pushing this ceiling—tighter separators, better boundary-table pruning, and the layout optimizer's crossing reduction—is the main empirical question this design opens.

Status of the secondary path. The stand-alone supergraph backend is retained only for theory and comparison. By Theorem 5.4 it is unbounded on general graphs, and it additionally requires iterated independent-set extraction to color on hardware. Its practically valuable contribution is its geometric machinery, which the primary path reuses as the layout optimizer.

11 Future Work

This paper presents the language and its certified-reduction type system (Section 8), backed by an axiom-free Coq mechanization of the core calculus (Section 9.1) and a working Haskell checker (Section 9.2). What remains is to grow this core into a *full* language with a complete metatheory: type preservation and progress for the surface elaboration itself—not only the reductions it emits—and a mechanized account of the grade lattice, the safety-effect monoid, and the split/stitch rules beyond the fragment now in Coq. Connecting the mechanized decoders to extracted, runnable code would close the gap between the verified calculus and the executable pipeline. Two engineering forks remain open: *which gadget set* to adopt (the verbatim construction of [37] versus a custom set, affecting what must be re-proved), and confirming the *weighted* MWIS interface on real hardware. While we already run the genuine `bloqade-analog` emulator on unit-disk layouts (Section 9.6), a full evaluation against QuEra-class parameters [48]—measuring atom count, area, crossing number, and ground-state success probability across structured and random instances at device scale—remains future work, gated by the size wall above.

12 Conclusion

Neutral-atom hardware solves UDG–MWIS, not coloring, and that single fact selects the architecture. Compiling coloring exactly means *reducing* it to UDG–MWIS via the classical

colorability→independent-set reduction composed with Rydberg copy/crossing gadgetization — a once-proved, inflation-free, polynomial pipeline with an exact decoder. The geometric heuristics that a supergraph approach would have leaned on for correctness are provably unable to bound chromatic inflation on general graphs; their proper and sound home is the layout optimizer that fits the exact gadget instance onto the device. This separation — exact correctness in the reduction, heuristic effort in the geometry — is the foundation on which a typed, certificate-carrying compiler can be built.

References

[1] Stefan Arnborg and Andrzej Proskurowski. Linear time algorithms for NP-hard problems restricted to partial k-trees. *Discrete Applied Mathematics*, 23(1):11–24, 1989.

[2] Massimo Bartoletti, Pierpaolo Degano, Gian-Luigi Ferrari, and Roberto Zunino. Local policies for resource usage analysis. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 31(6):1–43, 2009.

[3] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps and spectral techniques for embedding and clustering. *Advances in Neural Information Processing Systems*, 2003.

[4] Chiara Bodei, Pierpaolo Degano, Gian-Luigi Ferrari, and Corrado Priami. Comparing flow algebras for process calculi. In *Concurrency Theory (CONCUR)*. Springer, 1998.

[5] Hans L Bodlaender. A linear-time algorithm for finding tree-decompositions of small treewidth. *SIAM Journal on Computing*, 25(6):1305–1317, 1996.

[6] Immanuel M Bomze et al. The maximum clique problem. *Handbook of combinatorial optimization*, pages 1–74, 1999.

[7] Stephen Boyd and Lieven Vandenberghe. Convex optimization. *Cambridge University Press*, 2004.

[8] Daniel Brelaz. New methods to color the vertices of a graph. *Communications of the ACM*, 22(4):251–256, 1979.

[9] H Breu and DG Kirkpatrick. On the complexity of recognizing unit disk graphs. *Journal of Computational Geometry and Theory of Applications*, 1998.

[10] Marco Cerezo et al. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.

[11] Fan RK Chung. *Spectral Graph Theory*, volume 92. American Mathematical Society, 1997.

[12] Brent N Clark, Charles J Colbourn, and David S Johnson. Unit disk graphs. *Discrete Mathematics*, 86(1-3):165–177, 1990.

[13] WD Clark. Unit distance graphs with chromatic number four. *American Mathematical Monthly*, 97:184–191, 1990.

[14] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[15] Pierpaolo Degano, Gian-Luigi Ferrari, and Letterio Galletta. A two-component language for adaptation: design, semantics, and program analysis. *IEEE Transactions on Software Engineering*, 42(6):505–522, 2016.

[16] Gabriel Andrew Dirac. On rigid circuit graphs. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 25(1–2):71–76, 1961.

[17] Sepehr Ebadi et al. Quantum phases from competing short-and long-range interactions in an optical lattice. *Nature*, 595:227–232, 2021.

[18] Sepehr Ebadi, Alexander Keesling, Madelyn Cain, Tout T Wang, Harry Levine, Dolev Bluvstein, Giulia Semeghini, Ahmed Omran, J-G Liu, Rhine Samajdar, et al. Quantum optimization of maximum independent set using rydberg atom arrays. *Science*, 376(6598):1209–1215, 2022.

[19] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.

[20] Thomas MJ Fruchterman and EM Reingold. Graph drawing by force-directed placement. *Software: Practice and experience*, 21(11):1129–1164, 1991.

[21] Michael R Garey and David S Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[22] Fred Glover and Manuel Laguna. *Tabu search*. Kluwer Academic Publishers, 1997.

[23] Martin Charles Golumbic. Algorithmic graph theory and perfect graphs. *Academic Press*, 1980.

[24] TM Graham et al. Multi-qubit entanglement and algorithms on a programmable neutral-atom quantum processor. *Nature*, 604:457–462, 2022.

[25] Luc Henriet et al. Quantum computing with neutral atoms. *Quantum*, 5:528, 2021.

[26] A Heppes. On the packing density of circles and spheres. *Mathematika*, 2003.

[27] Daniel Jungck et al. On the complexity of udg recognition and related problems. In *Computational Geometry*, 2021.

[28] Richard M Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Springer, 1972.

[29] S Kirkpatrick, CD Gelatt, and MP Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[30] Joseph B Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29(1):1–27, 1964.

[31] Laszlo Lovasz. Kneser's conjecture, chromatic number, and homotopy. *Journal of Combinatorial Theory, Series A*, 34(3):319–324, 1983.

[32] John M Lucassen and David K Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 47–57. ACM, 1988.

[33] Aleksandr M Lyapunov. The general problem of the stability of motion. *International Journal of Control*, 55(3):531–534, 1907.

[34] Kent T Madsen et al. Scalable photonic platform for quantum information. *Nature*, 606:75–81, 2023.

[35] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. International Series in Operations Research & Management Science. Springer, 1999.

[36] Yurii Nesterov. Introductory lectures on convex optimization: A basic course. *Applied Optimization*, 87, 2004.

[37] Minh-Thi Nguyen, Jin-Guo Liu, Jonathan Wurtz, Mikhail D Lukin, Sheng-Tao Wang, and Hannes Pichler. Quantum optimization with arbitrary connectivity using rydberg atom arrays. *PRX Quantum*, 4(1):010316, 2023.

[38] Hannes Pichler, Sheng-Tao Wang, Leo Zhou, Soonwon Choi, and Mikhail D Lukin. Quantum optimization for maximum independent set using rydberg atom arrays. *arXiv preprint arXiv:1808.10816*, 2018.

[39] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[40] Fred S Roberts. Recent progresses in combinatorics. *Academic Press*, 1969.

[41] Neil Robertson and Paul D Seymour. Graph minors. II. algorithmic aspects of tree-width. *Journal of Algorithms*, 7(3):309–322, 1986.

[42] Mark Saffman, Timothy G Walker, and Klaus Molmer. Quantum computing with neutral atoms. *Reviews of Modern Physics*, 82(3):2313, 2010.

[43] Pascal Scholl et al. Quantum computing hardware for neutral atoms. *Nature*, 605:202–206, 2022.

[44] Robert E Tarjan. Decomposition by clique separators. *Discrete Mathematics*, 55(2):221–232, 1985.

[45] WT Tutte. On the chromatic polynomial. *Journal of Combinatorial Theory*, 2:145–155, 1954.

[46] DJA Welsh and MB Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–89, 1967.

[47] Douglas B West. *Introduction to Graph Theory*. Prentice Hall, 2nd edition, 2001.

[48] Jonathan Wurtz, Alexei Bylinskii, Boaz Braverman, Jesse Amato-Grill, Sergio H Cantu, et al. Aquila: QuEra's 256-qubit neutral-atom quantum computer. *arXiv preprint arXiv:2306.11727*, 2023.